

Monthly outcome: the robot measures the visible scene

M14 • 21 Nov 2027 - 19 Dec 2027 • 75 hours

Build a reproducible RGB-D processing pipeline: use colour images and depth to determine the geometry of a floor, a step and an object on a table. For a walking robot, this will provide heights and information about unknown regions ahead of a foothold; for an arm, it provides the position of a surface and a potential grasp object. The M14 deliverable contains calibration files, reference scenes, classical and learned methods, measurements of geometric error and latency, and a failure log.

What you need to understand before starting

Review coordinate transforms, rotation matrices, the 4×4 homogeneous matrix, measurement noise, NumPy, testing and ROS 2 recording from earlier modules. Check your understanding with a small task: transform a point from the camera frame into the body frame and back; the recovered point must match within the chosen numerical tolerance. Separately explain how a pixel differs from a metre and why the RGB timestamp may differ from the depth measurement time.

Working setup and boundaries

The main option is recorded RGB-D scenes or a simulator with known geometry; your own camera is useful but optional. Create three scenes: a flat surface, a box representing a step, and an object for a future grasp. TUM data is suitable for testing loading and geometry; prepare separate "support" and "obstacle" labels for your tasks. Do not attribute semantic annotations to the original dataset if it does not contain them.

Use Ubuntu 24.04 and ROS 2 Jazzy from previous months. Use Python for experiments, OpenCV and Open3D for geometry, and PyTorch for a small learning experiment. If necessary, port only a stage whose high computational cost has been demonstrated by measurement to C++. On the Mac, continue using the existing Linux environment. For every result, identify the CPU, architecture and accelerator used.

What understanding looks like

You can show how an incorrect depth scale distorts the estimated box height, and how a low error on training images can conceal poor calibration. Distinguish a recognised "floor" from geometrically verified support. Full SLAM, step selection and a real closed-loop walking controller belong to later stages. In M14, processing outputs describe only the observed scene and data reliability.

75 hours: work sequence and time accounting

Four work blocks

Hours 1-20: camera geometry, calibration, units and time; 10 hours of analysis and 10 hours of practice. Hours 21-40: colour, edges, features, RANSAC and tracking; 10+10 hours. Hours 41-60: point clouds, filtering, normals, ICP and heights; 10+10 hours. Hours 61-75: CNNs, transfer learning, comparison and acceptance; 5 hours of analysis and 10 hours of practice. Total: 35 hours of theory and analysis and 40 hours of practice, with no hours added beyond the programme.

Divide theory in each of the first three blocks into five 2-hour sessions; divide the final 5 hours as 2+2+1. Organise practice into four Sunday blocks of 10 hours, with breaks outside study time. A practical block may be split across available intervals with the same total duration. Roadmap calendar boundaries may overlap between neighbouring modules; retain rest, holidays and separate integration checkpoints without counting hours twice.

One experiment, one testable version

At the start of a session, record a question such as "How does voxel size affect estimated step height?" Then identify the parameter to change, the fixed input dataset and the expected sign of success. At the end, save the configuration, dataset hash, plot and failure explanation. An experiment that changes calibration, image size and network weights simultaneously cannot establish the effect of one factor.

Organise the project as follows: calibration/ - K, distortion and procedure; datasets/manifest - data provenance and splits; geometry/ - depth processing; perception/ - masks; tests/scenes - reference scenes; reports/ - metrics. Store large raw recordings separately from code, preserving exact names, checksums and acquisition instructions. The entire dataset must be reprocessed with one command, without manually selecting successful frames.

Working as an experienced developer

Read until you can answer the current question, then move to measurement. For each new algorithm, work through a few points by hand, build a small test and only then process a full recording. "It did not work" is insufficient: identify the stage, input, version and symptom. If package installation exceeds its allotted time, replay a prepared recording and reduce the experiment's scale; retain calibration and unit checks.

The camera: from pixel to geometric ray

Topics to study

Study the pinhole model: $u = f_x \cdot X/Z + c_x$, $v = f_y \cdot Y/Z + c_y$. Coordinates X, Y, Z are in metres in the optical frame; u, v and f_x, f_y, c_x, c_y are in pixels. Matrix K describes the intrinsic parameters. Radial and tangential distortion alter observed pixels; the extrinsic transform relates the camera to the body. Do not confuse lens calibration with the camera's pose on the robot.

Study stereo depth using $Z = f \cdot b/d$: focal length f is in pixels, baseline b in metres and disparity d in pixels. At small disparity, the same pixel error produces a larger depth error. Compare two distances with the same error in d : a distant surface is less suitable for precise foothold selection. For RGB-D, understand scale, channel alignment and the representations of missing or invalid depth values. Hand-eye calibration relates camera and link motions: study $AX=XB$ and the eye-in-hand/eye-to-hand distinction. M14 requires a measurement scheme and understanding of the necessary extrinsics; a separate solver is not required.

Task 1. Check the camera model

Input: synthetic 3D points in front of the camera and a specified K . Project the points, recover X, Y from known Z values, then test the image centre, doubling Z and a point behind the camera. Output: a projection-and-reconstruction round-trip test and a drawing of the rays. For synthetic data, specify a relative numerical tolerance, such as 10^{-6} , as a computational requirement. $Z \leq 0$ and missing depth must cause rejection rather than produce a point at the origin.

Task 2. Calibrate for the working field of view

Use at least 15-20 varied chessboard images: include image edges, different tilts and distances. If you have no camera, use published OpenCV images; document square size as either a known value from the source or an arbitrary unit. Split images by capture pose into a parameter-fitting set (fit) and a held-out set. Find and refine internal corners, estimate K and distortion parameters, and save a list of excluded images with reasons.

Check by overlaying reprojected points, calculating RMS pixel distances and reporting each image's errors separately. On a held-out image, estimate only the board's position and orientation while keeping K and distortion fixed. Repeat using only central views and compare edge errors. Document all checks; one final number is insufficient. A small RMS does not establish correct physical scale or the correct camera-to-body transform.

Classical vision: a baseline and its limits

Sections to cover

BGR/RGB and HSV colour spaces; smoothing, gradients and Canny; binary masks, morphology and connected components; local features and matching; geometric models and outliers. Separate the signal from the decision: an image edge is not the edge of a step expressed in metres. A homography is sufficient for a textured planar surface; one homography cannot describe an arbitrary 3D scene.

Task 3. Find an object and a surface edge

Input: 30 images from three scenes, with a manual object mask and marked edge. Build an HSV mask, then an edge-based alternative; fix thresholds using the training portion. Remove small components and save the object contour and bounding rectangle. Output: masks, overlays and a detection-results table. Evaluate IoU = intersection area / union area on the same dataset, and the fraction of images with no detection.

Change lighting, add a similarly coloured object, then add a shadow on the plane. Do not adjust a threshold to fix one image after inspecting the test set. A false object detection beside the arm and a missed edge ahead of a leg have different consequences; annotate these cases in the report. Separately test an empty mask, an entirely white mask and an object cropped by the image boundary.

Task 4. Robust matching and tracking

Find features in a pair of images of a flat board, match descriptors and estimate a homography using RANSAC. Introduce a controlled fraction of deliberately incorrect pairs. Save the number of matches and model-consistent matches (inliers), the geometric error of reference points and an "insufficient data" flag. Test a degenerate, nearly collinear configuration and a repeated pattern. Do not accept a matrix simply because the function returned one.

Next, run Lucas-Kanade on selected corners in a short video: compare forward and backward motion and reject unreliable point tracks. Save the tracks and a plot of the number of points that remain tracked. Blur or switching off the light must reduce confidence in the estimate. Tracking establishes correspondence between images; without geometry and depth, it does not yet provide metric body velocity. Save original images and the RANSAC seed so that repeated runs are comparable.

Point clouds: the surface in the robot frame

What to study and check by hand

For an aligned depth map: $Z = \text{depth}/\text{scale}$, $X = (u - c_x) \cdot Z/f_x$, $Y = (v - c_y) \cdot Z/f_y$. First exclude missing and invalid values. Transform into the body frame with $p_{\text{body}} = R_{\text{body_camera}} \cdot p_{\text{camera}} + t_{\text{body_camera}}$. Record the direction of every transform and check which inverse is required. Optical axes differ from body axes, so check labelled basis vectors and a point directly in front of the camera.

Filtering includes cropping to the working region, voxel-grid downsampling and outlier removal. A normal describes the local surface direction; neighbourhood radius affects results near an edge. For a plane $n \cdot p + d = 0$, normalise n to unit length so that the residual is in metres. ICP reduces correspondence error with a suitable initial estimate, but does not guarantee correct registration of every scene.

Task 5. From depth to a plane and a step

Use a synthetic scene with a known box height, then a real recording. Convert depth to points, crop the area in front of the body and compare three voxel sizes. Use RANSAC to extract a support plane, estimate normals and build a height grid with an observation count per cell. Output: a point cloud, plane.yaml, a height map, an unknown-cell map and height error over reference regions.

Check that synthetic height is recovered within a tolerance chosen beforehand; for recordings, use a measured reference and its uncertainty. Mark empty cells unknown; do not substitute zero. Add 10% random outliers and a region of missing depth at the edge. Compare median absolute height error and the fraction of cells without an estimate. Choose the threshold in metres to reflect scene scale and sensor noise.

Task 6. Register adjacent observations

Create a pair of clouds with a known translation and rotation. Run point-to-point and then point-to-plane ICP from identical initial conditions. Report transform error, correspondence fraction, residuals and time; classify small residuals with an incorrect displacement as a failure. Repeat on a single plane, in a symmetric corridor and with a poor initial estimate. Explain how incorrect registration would shift an object for the arm and create a false support height for the leg.

Learned vision: a small comparison experiment

The minimum theory for the week

Study convolution, receptive fields, feature maps, backbone weights and the output head. Learn training and inference, train/eval modes, transfer learning, learning rate and overfitting. The aim is to design an experiment without developing a large network. Compare methods on the same task and annotations: for example, segment a specified class of object on a table or a region of flat surface.

Image classification, bounding-box detection and pixel segmentation produce different outputs. Choose a compact segmentation model with a pretrained backbone and fine-tune only a small head, or use a ready-made detector for a supported class. In the latter case, perform a separate transfer-learning experiment on a small classifier of cropped image patches; do not describe an unchanged ready-made model as fine-tuned. Limit annotation of your own scenes to 60-100 temporally separated images.

Task 7. Train without data leakage

Split data by scene or capture session rather than randomly distributing neighbouring images. Initial split: train 60%, validation 20%, test 20%. Save each file's assignment and check overlaps using hashes and timestamp proximity. Apply augmentations only to train; transform images and masks consistently for geometric changes. Do not flip a scene vertically unless those conditions are physically possible in the intended setting.

Limit head-training time, save the best version according to the validation set and fix the threshold. Prepare weights, preprocessing parameters, the training log and manifest. Check train/eval modes on the same input; during inference, results must not depend on test-image order. Explicitly define behaviour when the required class is absent or the image is corrupted.

Task 8. Quality, confidence and latency

Compare results with Task 3 on the test set using precision, recall and IoU. State the counting unit: a pixel or a detected object. For five confidence bins, tabulate mean model confidence and the actual fraction of correct results; note the small sample size. Measure classical and learned method latency over the same complete processing segment. A CNN may improve recall yet fail on a new material; reflect this in the report. Permit unknown outside the studied conditions so uncertainty does not become a robot command.

Integration laboratory: surveying terrain and a table

Specification

The replay command accepts an RGB-D manifest, calibration.yaml, frames.yaml and method parameters. For each image it outputs the measurement timestamp, frame_id, a valid-depth mask, a plane, a height grid, an object mask, diagnostic status and processing time. Add a small ROS 2 message adapter to the existing project. The computational core must process recordings without a connected robot. Preserve the original timestamp; do not replace it with processing-completion time.

Assembly sequence

1. Load an RGB/depth pair and check size, encoding and scale.
2. Match timestamps within a specified tolerance and reject unpaired images.
3. Apply the correct calibration; update K consistently when image size changes.
4. Generate points and transform them into the body frame.
5. Compute geometry and the semantic mask independently.
6. Combine results in a report while preserving reasons for unknown values.
7. Check reproduction with one command.

Task 9. Verify what the outputs mean

In scene A, the robot looks at a level floor; in B, a box is ahead; in C, the camera observes an object on a table. For each, define 5 reference regions with known roles: floor, box top, edge, a region without depth data and background. Compare height, normal and the fraction of valid measurements. Output: a region table, overlays and an explanation of where the network disagrees with geometry. Do not allow a "floor" mask to remove a geometric obstacle automatically.

For future foothold selection, propose only candidate regions: visible flat cells with estimated heights. Leg reachability, load-bearing capacity, friction and stability are not checked here. For the arm, propose the observed object surface; grasp orientation and collision freedom will be checked in M17. This establishes which conclusions the data supports.

What to include in the deliverable

Save a README with commands, environment and library versions, input data or acquisition instructions, calibration and the coordinate-frame chain, a table of all scenes, numerical metrics, at least six failure or boundary cases and usage limitations. Add a short replay video labelled with time and coordinates. Check results against raw arrays: video compression may conceal dropped frames, so video does not replace the log.

Error checks: eight mandatory scenarios

A set of controlled faults

S1 - Incorrect depth scale: multiply distances by 1000 while retaining the settings. A range check or comparison with reference geometry must detect the error; record the stage that detects it. S2 - Incorrect K after resizing: halve the image size while keeping the original K. Reference height or projection must reveal a discrepancy. Restore a consistent transform and save both results.

S3 - Time offset: pair RGB and depth with offsets of 0, 1 and 3 frames in a moving scene. Measure mask-boundary error relative to the cloud and the number of rejected pairs. S4 - Missing depth at a potential foothold: zero depth in a square beneath the future leg position; the output must contain unknown. Do not increase map completeness by interpolating across an unknown edge without labelling that interpolation separately.

S5 - Geometric degeneracy: feed ICP a single plane translated along itself. Compare position-and-orientation error with the residual; explain the lack of constraints. S6 - Changes in lighting and material: compare the classical method and network on a fixed test set. Show false detections, misses and model confidence. Either method may lose the comparison, provided experiment parameters were fixed beforehand.

S7 - Data leakage: deliberately add a copy of a training image to test; the automatic manifest check must fail before quality metrics are calculated. S8 - Overload: add delay to network inference, then inspect counts of received, processed and discarded images. Publishing an old map must preserve its age and diagnostic status rather than present it as fresh.

Units and metrics

Reprojection error is the RMS of Euclidean distances in pixels; do not mix it with mean norm/N from another example. Height error is the absolute difference in metres at reference points. Measure computation time using the end and start readings of one monotonic timer; data age requires aligned clocks. Report median, p95, maximum and image count. Measure file loading and warmed-up network inference separately.

Documenting a failure

Each scenario stores its original configuration, the single introduced change, the expected observation and the actual outcome. If a check misses S1 or S4, refine data requirements before improving the CNN. With a physical camera, separately record lighting, mounting and reference conditions: a synthetic tolerance does not automatically become an acceptable error for the real robot.

Sessions 1-40: the camera and classical method

Hours 1-10: five 2-hour sessions

1-2: understand K, projection and units; calculate the centre and one off-axis point by hand, then work on Task 1. 3-4: radial and tangential distortion and the meaning of internal board corners; prepare the capture procedure. 5-6: extrinsics, optical frame and body-frame transform; check three axes. 7-8: depth, stereo baseline, scale and uncertainty at two distances. 9-10: reprojection criterion, image splits and expected Task 2 errors.

Hours 11-20: first practical, 10 hours

11-12: prepare 15-20 images and a manifest; check provenance and usage rights. 13-14: find and refine corners, then visually check their order. 15-16: calibrate and estimate the held-out board's position and orientation; save parameters. 17-18: plot residuals by image and compare central-view and varied-view sets. 19-20: prepare recording reprocessing, a README and the first report. By the end of the block, calibration must be reproducible from saved data.

Hours 21-30: five 2-hour sessions

21-22: colour and preprocessing; explain the effect of BGR/RGB on HSV. 23-24: edges, morphology, connected components and IoU. 25-26: features, descriptors and nearest neighbours; record indicators of incorrect matches. 27-28: RANSAC and the geometric model; draw an example with outliers. 29-30: optical flow, violations of brightness constancy and a track-rejection criterion. Prepare a specific input for the next practical during every session.

Hours 31-40: second practical, 10 hours

31-32: annotate reference scenes and implement the Task 3 HSV and edge baselines. 33-34: evaluate quality, fix thresholds, change lighting and add a similar object. 35-36: complete Task 4 with features, homography and outliers. 37-38: track points through a short sequence, check forward/backward consistency and failure when images are lost. 39-40: repeat the dataset with a fixed seed, check output units and prepare failures.md.

Even if you already know a topic from software development, check it in the robotics context. Use the time saved for an additional reference viewpoint and failure explanation. Do not add a separate web interface: an error plot and an image with labelled axes are more useful at this stage.

Sessions 41-75: 3D, the network and the final package

Hours 41-50: five 2-hour sessions

41-42: XYZ reconstruction and masks for missing and invalid values; check 3 pixels by hand. 43-44: voxels, outliers and normals; compare the scales needed for a floor and a small object. 45-46: RANSAC planes, normalised residuals and height grids. 47-48: ICP, initial estimates, point-to-point/point-to-plane variants and degeneracy. 49-50: reference selection, transform error, applicability limits and acceptable processing latency.

Hours 51-60: third practical, 10 hours

51-52: Task 5, a synthetic cloud and a recording with known scale. 53-54: plane, height map and unknown; compare three voxel sizes. 55-56: Task 6 with a known displacement and two neighbouring images. 57-58: test a plane without texture or structure, an incorrect transform and areas without depth. 59-60: assemble the geometry part of Task 9, the report and a fixed interface for the next month.

Hours 61-65: theory 2+2+1

61-62: CNN features, backbone and head, train/eval modes and transfer learning; select a small model. 63-64: session-based data splits, augmentation, precision/recall/IOU and confidence calibration. 65: formulate the comparison hypothesis, training-time limit and stopping criterion. Check that the data contains object or floor labels. If not, choose a class you can annotate correctly.

Hours 66-75: fourth practical, 10 hours

66-67: Task 7, manifest data splits and minimal head fine-tuning. 68-69: Task 8, a fixed test set, classical-versus-network comparison and five confidence bins. 70-71: Task 9, a combined terrain-and-table report and stage timing. 72-73: scenarios S1-S8, leakage checking and recording reprocessing. 74-75: acceptance and handover to M15. If training does not finish within the allocated time, state that limitation; preserve the baseline and checks.

Reducing scope if you fall behind

First remove a second network architecture, then detailed ICP tuning and porting to C++. Retain one learned baseline, a comparable pair of geometric methods, calibration, the cloud, the height map and the full failure suite. Save time by reducing dataset size and the number of parameters; retain a separate test set. Rust is only an optional exercise beyond the required deliverable here, without a third implementation of the pipeline.

Acceptance checks and the transition to localisation

A verifiable monthly deliverable

Include K , the distortion model, image size, physical or arbitrary board scale, original images, a held-out set and errors by viewpoint in the calibration package. Specify time, scale and pairing rules in the RGB-D manifest. Define transform directions and units for the frame chain; check forward and inverse conversion. Saved data must be sufficient to reproduce setup without verbal explanations from its author.

The pipeline builds a cloud, extracts a plane and object, estimates heights, preserves unknown and emits diagnostics. It includes at least one classical and one learned baseline, a shared test set and comparisons of quality and latency. Scenarios S1-S8 must be reproducible, rather than only described. Report limitations of the real sensor if one was used; clearly separate synthetic results from physical measurements.

Self-check without notes

1. Why is f_x measured in pixels, and what changes when image size changes?
2. How does a scale error differ from reprojection error?
3. Why must zero depth not be treated as a floor point?
4. How do you check the direction of $T_{\text{body_camera}}$?
5. When is a homography valid, and when is it not?
6. Why can ICP with a small residual give an incorrect position and orientation?
7. How does voxel size affect a step edge?
8. Why do neighbouring images in training and test sets create leakage?
9. What does high recall with low precision mean for the arm?
10. How does model confidence differ from the actual probability of error?
11. Why must a measurement timestamp not become a publication timestamp?
12. What additional checks are needed before allowing the real robot to place a foot in a selected cell?

What to pass to M15

Pass on short synchronised sequences, camera parameters, the coordinate-frame chain, ICP settings, reference displacements and descriptions of degenerate cases. Preserve reasons for lost point tracks and sensor dropouts: they become localisation test scenarios. Explicitly identify artificially corrected trajectories and excluded failed images.

The final demonstration takes 5-7 minutes: one normal scene, incorrect depth scale, a region without depth, a method comparison and a repeated launch. Separately state what the project does not yet measure: support load-bearing capacity, physical friction, actual contact and step stability. The next stage can use the described geometry, but must make motion decisions under its own constraints.

Assigned reading: a focused route through the sections

The literature is mainly in English; the programme and definitions are also in English. Read the selected sections before the relevant task, rather than entire books. Core reading is included in the existing 35 analysis hours; consult references as needed.

Core reading

R. Szeliski, Computer Vision: Algorithms and Applications, 2nd ed. (2022). [Author's page](#). Ch. 2 Image Formation; selected parts of Ch. 7 Feature Detection and Matching and Ch. 9 Motion Estimation. For Tasks 1-4: camera rays, robust correspondences and Lucas-Kanade; record the conditions under which a method ceases to be meaningful.

R. Hartley, A. Zisserman, Multiple View Geometry in Computer Vision, 2nd ed. (2004). [Book and authors' contents](#). Ch. 6 Camera Models; only two-view geometry from Ch. 9 Epipolar Geometry and the Fundamental Matrix; triangulation from Ch. 12 Structure Computation. Before Tasks 1-2, learn to distinguish calibration, baseline, depth and uncertainty.

For the small deep-learning experiment

I. Goodfellow, Y. Bengio, A. Courville, Deep Learning (2016). [Authors' online book](#). Ch. 9 Convolutional Networks: convolution and pooling; Ch. 11 Practical Methodology: metrics and model selection. Before Tasks 7-8, understand the backbone and head, data splitting and the experiment's time limit; the remaining chapters are not needed yet.

Three implementation references

[OpenCV: Camera Calibration](#) - Calibration, Undistortion, Re-projection Error; Task 2, fixed K and distortion with separate held-out images.

[Open3D: ICP registration](#) - Point-to-point and Point-to-plane ICP, evaluation. In Task 6, compare the residual with the error of the known transform.

[PyTorch: Transfer Learning](#) - ConvNet as fixed feature extractor and Train the model; Task 7, a small CPU experiment with saved preprocessing settings and weights.

After reading, explain why a small reprojection error does not establish scale, why ICP can fail despite a small residual, and why model confidence does not equal the probability of a correct decision without checking calibration.

Terms and definitions

Pinhole projection. An ideal camera model in which image coordinates are determined by the ratios of a point's transverse coordinates to its depth.

Intrinsics. The focal scales and principal point in K ; they determine the mapping from camera rays to pixel coordinates.

Extrinsics. Rotation and translation between the camera and another coordinate frame, with an explicitly stated transform direction.

Distortion. A systematic deviation from the selected ideal projection caused by optics and described by a calibration model.

Reprojection error. The distance between a measured pixel and the projection of an estimated point; usually expressed in pixels of the selected image.

Disparity. The coordinate difference between corresponding points in a rectified stereo pair; together with baseline and focal length, it determines depth.

Stereo baseline. The distance between camera optical centres in metres; it establishes the metric scale of stereo depth.

Depth scale. A factor converting a stored depth code into physical units; the data format determines the direction of conversion.

Homography. A projective mapping of a plane between images; one homography is generally insufficient for a scene with different depths.

Feature descriptor. A vector describing a local image region for matching; descriptor similarity requires subsequent geometric verification.

RANSAC. A model-estimation method using random minimal subsets, checking consistency of the remaining data and rejecting outliers.

Inlier. An observation whose model residual is within a threshold; this does not yet establish that the correspondence is correct.

Optical flow. A field of apparent image displacements between times; by itself, it does not define metric body velocity.

Point cloud. A set of spatial coordinates of measured or reconstructed surfaces, expressed in a common frame and physical units.

Voxel downsampling. Reduction of cloud density by combining points within spatial cells; cell size determines the loss of detail.

Surface normal. A direction perpendicular to the local tangent plane; its estimate depends on the neighbourhood and may be ambiguous at an edge.

ICP. Iterative refinement of a transform through cloud correspondences; it requires suitable initialisation and may converge to an incorrect solution.

Segmentation. Assignment of a label to each pixel; distinct from whole-image classification and detection of rectangular object regions.

Transfer learning. Use of pretrained weights while adapting selected layers to a new task and dataset.

IoU. The ratio of intersection area between predicted and reference regions to their union area; the rule for empty regions is specified separately.

Equipment and setup for this month

Use existing equipment

MacBook Pro M4, Jazzy in UTM, M13 data and geometry, a ruler, callipers and mounting hardware. Recorded RGB-D and synthetic data are suitable for algorithm work. [TUM RGB-D: File Formats](#) specifies depth scale, timestamps and the ground-truth format; saved images can be analysed on macOS.

Purchase before recording your own data

Select 1 RGB-D camera providing metric depth and RGB, accessible intrinsics and extrinsics, timestamps, a mask of missing and invalid depth, and a driver for the selected Jazzy/Ubuntu ARM64 environment. Orbbec Gemini 2 is one candidate whose compatibility still requires confirmation. Before ordering, check the exact model, firmware, SDK branch and ARM64 support against the [official Orbbec ROS 2 table](#). Separately verify USB in UTM using an available sample or a confirmed configuration.

Prepare 1 suitable USB data cable and 1 stable mount; use a powered hub only if testing establishes a need. Make 1 flat calibration board and measure its square size. Choose range and field of view for the workbench and scene, rather than maximum specification values. If compatibility is unconfirmed, continue with recordings and defer the purchase.

Install now

Install OpenCV and Open3D in a separate Python environment. [Open3D: installation and platforms](#) - select a wheel for your Python version and architecture; open3d-cpu targets x86_64 Linux. If no suitable Linux ARM64 wheel exists, analyse saved clouds natively on macOS ARM; a source build requires separate verification.

In the Ubuntu guest, configure the camera ROS driver and USB access according to the specific SDK instructions. For Task 7, add PyTorch/torchvision CPU and Jupyter in a separate environment; fix a small backbone, head and weights. The Mac M4 is not a CUDA device; purchasing a GPU or Jetson is not required here.

Check the installation

Import cv2, open3d and torch; read an RGB/depth pair, check scale against a measured plane and build a cloud. The camera must provide real, non-empty images, CameraInfo and consistent timestamps, and USB recording must be stable. Test ICP on a known transform and run the network once to verify its expected output shape. Setup is included in the 75 hours; delivery is outside study time.

Motion fundamentals: from pixel to target

Required: why a visible point is not yet a leg target

A pixel defines a ray rather than a unique point: without depth, distance along that ray remains unknown. After reconstructing XYZ, you need the camera's pose on the robot and the body's position and orientation at image time. Check the prerequisite: explain the difference between rotating a vector and translating a point, and label the optical axes C and body axes B.

Model and worked example

Use a rectified image without distortion. u, v are pixels; f_x, f_y, c_x, c_y are parameters in pixels; Z is depth along the optical axis in metres, not range along the ray. Then $X = (u - c_x)Z/f_x$, $Y = (v - c_y)Z/f_y$. With $f_x = f_y = 600$, $c_x = 320$, $c_y = 240$, $(u, v) = (370, 240)$ and $Z = 1.2$ m, we obtain $p_C = (0.1; 0; 1.2)$ m.

For an error of 1 pixel in u and exactly known Z , the transverse error is $\delta X = Z \cdot \delta u / f_x = 0.002$ m. A coordinate-frame rotation preserves the length of this error vector; depth and extrinsic-parameter errors are added separately. This is how pixel noise becomes uncertainty in a future movement.

In this example the camera faces forwards: its X points right, Y down and Z forwards; B has x forwards, y left and z up. Therefore $R_{BC}(X, Y, Z) = (Z, -X, -Y)$. The camera centre is $t_{BC} = (0.2; 0; 0.3)$ m: $p_B = R_{BC} p_C + t_{BC} = (1.4; -0.1; 0.3)$ m. Thus the object is to the right of the body, not the left; camera depth is not height above the floor.

At image time, let the body have $\text{yaw} = \pi/2$ and $p_{WB} = (1; 2; 0)$ m in world frame W . Then $p_W = R_{WB} p_B + p_{WB} = (1.1; 3.4; 0.3)$ m. In homogeneous notation, $T_{WC} = T_{WB} T_{BC}$: the rightmost transform acts first. If the camera is on a moving link, T_{BC} depends on the joints; when the body moves, use transforms at measurement time.

Two exercises within the existing laboratory

1. In Task 5, save three points: on the optical axis, to its right and below it. Draw frames C, B and W and tabulate coordinates; check the inverse path $W \rightarrow C$ and reprojection. For synthetic data, set computational tolerances of 10^{-6} m and 10^{-6} pixels. $Z \leq 0$ and missing or invalid depth must cause rejection; they are not points at the origin.

2. Keep the object fixed and move the body 0.1 m along its x -axis without rotation. In C, Z is now 1.1 m and $u \approx 374.55$; the world point must remain unchanged. Substitute the old body position and orientation and demonstrate a 0.1 m error. Save replays of both variants and explain how a timing error displaces a proposed grasp point or foothold; reachability and load-bearing capacity are checked later.

Time accounting: 1 hour from sessions 5-6 for coordinate frames and formula derivation; 2 hours from 51-52 for these checks as a concrete implementation of Task 5. This remains within the original 75 hours, with no new equipment or separate project.

Assigned reading: Lynch/Park, [Modern Robotics §3.3.1](#) - composition and inverse transforms; Tedrake, [Geometric Pose Estimation: Representations for geometry](#) - depth, clouds and coordinate frames. Compare the camera equations with Task 1.